

UNIVERSITY OF BOLOGNA

Master's degree in Mathematics

**Theory and application of
central FD method applied to
Diffusion-Convection-Reaction
PDE**

Authors:

Natalie Elena Cernei

Alberto Cozzani

Niccolò Chiari

ACADEMIC YEAR 2023/2024

Contents

Introduction	ii
1 Central FD for Diffusion-Convection-Reaction	1
1.1 Computation of the matrix A	2
1.2 Accuracy and stability:	5
1.2.1 Accuracy	6
1.2.2 Stability	7
2 A brief guide on how the app works	10
2.1 First modality:	10
2.2 Second modality:	14
3 Conclusions	17

Introduction

The diffusion-convection-reaction problem is an elliptic partial differential equation, it is a combination of the diffusion and convection (advection) equation, and describes physical phenomena where particles, energy are transferred inside a physical system due to two processes: convection and advection.

The general equation is:

$$-\nabla \cdot (k \nabla u) + \mathbf{b} \cdot \nabla u + qu = f, \quad (x, y) \in \Omega \quad (1)$$

where:

- k describes the diffusion;
- $\mathbf{b} : \mathbb{R}^2 \rightarrow \mathbb{R}^2$ describes convection (or advection), $\mathbf{b} = (\mathbf{b}_1, \mathbf{b}_2)$;
- q is the reaction term.
- f is the known data;
- u is the unknown function we need to find.

To simplify the notation we will consider the diffusion coefficient k always equal to 1.

The goal of the chosen project is to develop a Matlab APP that enables the creation of the convergence and conditioning number plots, as well as visualizations of the numerical solution in two distinct methods; The first one will allow the user to specify the exact solution u of the problem while in the second version the user will be able to assign the function f .

The numerical solution will be computed using the Central Finite Difference method, incorporating mixed boundary conditions on a rectangular domain Ω .

In both modalities the parameters $\mathbf{b}$, q will be chosen by the user along with boundary conditions and domain specification.

Chapter 1

Central FD for Diffusion-Convection-Reaction

Consider the problem:

$$\begin{cases} -\Delta u + \mathbf{b} \cdot \nabla u + qu = f, & (x, y) \in \Omega, \quad q \in \mathbb{R} \\ u = g, & (x, y) \in \Gamma_D \\ \frac{\partial u}{\partial \vec{\nu}} + ru = h, & (x, y) \in \partial\Omega \setminus \Gamma_D, \quad r \in \mathbb{R} \end{cases} \quad (1.1)$$

where $\Omega = [a, b] \times [c, d]$ and $\mathbf{b} = (b_1, b_2)$.

We'll sometimes use the notation

$$\partial\Omega = \Gamma_L \cup \Gamma_B \cup \Gamma_R \cup \Gamma_T$$

with where $\Gamma_L, \Gamma_B, \Gamma_R, \Gamma_T$ are respectively the left, the bottom, the right and the top sides of the rectangle domain Ω . Discretize the domain Ω using grid points (x_i, y_j) with $i = 0, \dots, N+1$ over the x -direction and $j = 0, \dots, M+1$ over the y -direction, set $x_i = a + ih_x$ and $y_j = c + jh_y$, $h_x = \frac{b-a}{N+1}$ and $h_y = \frac{d-c}{M+1}$.

Denoting $u_{i,j} \approx u(x_i, y_j)$ the numerical solution of (1.1), we have:

- $\frac{\partial u_{i,j}}{\partial x} = \frac{u_{i+1,j} - u_{i-1,j}}{2h_x}$ and $\frac{\partial u_{i,j}}{\partial y} = \frac{u_{i,j+1} - u_{i,j-1}}{2h_y}$;
- $\frac{\partial^2 u_{i,j}}{\partial x^2} = \frac{u_{i+1,j} - 2u_{i,j} + u_{i-1,j}}{h_x^2}$ and $\frac{\partial^2 u_{i,j}}{\partial y^2} = \frac{u_{i,j+1} - 2u_{i,j} + u_{i,j-1}}{h_y^2}$.

Substituting inside (1.1) we obtain:

$$\begin{aligned}
 & -\left(\frac{u_{i+1,j} - 2u_{i,j} + u_{i-1,j}}{h_x^2} + \frac{u_{i,j+1} - 2u_{i,j} + u_{i,j-1}}{h_y^2}\right) + \\
 & \mathbf{b} \cdot \left(\frac{u_{i+1,j} - u_{i-1,j}}{2h_x}, \frac{u_{i,j+1} - u_{i,j-1}}{2h_y}\right) + qu_{i,j} = f_{i,j}
 \end{aligned} \tag{1.2}$$

1.1 Computation of the matrix A

Firstly, we are going to compute the matrix A in the case of (1.2) with homogeneous Dirichlet boundary condition, $g = 0$ on $\partial\Omega$. The associated discretized problem (1.2) can be written in matrix form as $AU = F$

$$U = \begin{bmatrix} u^{[1]} \\ u^{[2]} \\ \vdots \\ u^{[M]} \end{bmatrix}, \text{ where } u^{[j]} = \begin{bmatrix} u_{1,j} \\ u_{2,j} \\ \vdots \\ u_{N,j} \end{bmatrix} \text{ and } F = \begin{bmatrix} f^{[1]} \\ f^{[2]} \\ \vdots \\ f^{[M]} \end{bmatrix}, \text{ where } F^{[j]} = \begin{bmatrix} f_{1,j} \\ f_{2,j} \\ \vdots \\ f_{N,j} \end{bmatrix}$$

the general structure of the matrix A depends on the ordering of the unknowns U , as can be seen above, our choice is the natural row-wise ordering. For the computation of the matrix A we will use the Kronecker product. Let's consider the following matrices:

- D_{xx} the matrix associated to the central second order FD in the x -direction

$$D_{xx} = \begin{pmatrix} -2 & 1 & & & \\ 1 & -2 & 1 & & \\ & 1 & -2 & \ddots & \\ & & \ddots & \ddots & 1 \\ & & & 1 & -2 \end{pmatrix}$$

Equivalently D_{yy} is the matrix associated to the central second order FD in the y -direction, computed as D_{xx} ;

- D_x the matrix associated to the central first order FD in the x -direction

$$D_x = \begin{pmatrix} 0 & 1 & & & \\ -1 & 0 & 1 & & \\ & -1 & 0 & \ddots & \\ & & \ddots & \ddots & 1 \\ & & & -1 & 0 \end{pmatrix}$$

CHAPTER 1. CENTRAL FD FOR DIFFUSION-CONVECTION-REACTION3

Equivalently D_y is the matrix associated to the central first order FD in the y -direction, computed as D_x ;

- I_x, I_y identity matrices;

For the moment the boundary conditions associated to our problem are homogeneous Dirichlet, so the matrices associated to the FD discretization along x -direction will have dimension equal to $N \times N$, the ones associated to the y -directions will have dimension equal to $M \times M$;

- I the sparse identity matrix, $I \in \mathbb{R}^{(N \cdot M) \times (N \cdot M)}$

Now we can define:

- $K_1 = \frac{I_y \otimes D_{xx}}{h_x^2} \quad K_2 = \frac{D_{yy} \otimes I_x}{h_y^2}.$
- $K_3 = \mathbf{b}_2 \frac{D_y \otimes I_x}{2h_y} \quad K_4 = \mathbf{b}_1 \frac{I_y \otimes D_x}{2h_x}$

with $(\mathbf{b}_1, \mathbf{b}_2)$ components of the vector $\mathbf{b}$. In this way we are able to construct the matrix A as:

$$A = -(K_1 + K_2) + (K_3 + K_4) + qI.$$

We need to study also the cases in which the Dirichlet is not homogeneous and consider how it changes the computation of A .

Consider the non-homogeneous Dirichlet condition on the whole boundary, i.e. $g \neq 0$, call constrained nodes the mesh points on $\partial\Omega$ where we'll apply the Dirichlet conditions and free nodes the remaining ones. Define the boundary observation function R_g as:

$$R_g(x, y) = \begin{cases} g(x, y) & \text{if } (x, y) \text{ is a constrained node} \\ 0 & \text{if } (x, y) \text{ is a free node} \end{cases}$$

Then we decompose the solution u as the sum of its components in the free and constrained nodes:

$$u = v + R_g, \quad v = 0 \text{ in } \partial\Omega$$

where v satisfies the same problem with homogeneous Dirichlet conditions:

$$\begin{cases} -\Delta u + \mathbf{b} \cdot \nabla u + qu = f, & (x, y) \in \Omega, \quad q \in \mathbb{R} \\ v = 0, & (x, y) \in \partial\Omega \end{cases} \quad (1.3)$$

Applying the FD discretization to this last problem, we obtain exactly (1.2) that is equivalent to the linear system $AU = F$, substituting U :

$$A(V + RG) = F, \text{ where } V = \begin{bmatrix} v^{[1]} \\ v^{[2]} \\ \vdots \\ v^{[M]} \end{bmatrix} \text{ and } RG = \begin{bmatrix} RG^{[1]} \\ RG^{[2]} \\ \vdots \\ RG^{[M]} \end{bmatrix}$$

Defining the vector **free_nodes** of the indices of the free nodes, we compute v addressing only the free nodes in the matrix and r.h.s:

$$A(\mathbf{free_nodes}, \mathbf{free_nodes}) V = F(\mathbf{free_nodes}) - Z(\mathbf{free_nodes}) \quad (1.4)$$

where:

$$Z \equiv F - A \cdot RG \quad (1.5)$$

We are working with Dirichlet and Robin boundary conditions and since we do not know whether the conditions are applied to the left, bottom, right or top side, we can study a particular case in which the Robin conditions are applied just on the left side of the rectangular domain. All of the four sides can be treated in the same way except for the four corners that have to be computed apart.

The Robin conditions are:

$$\frac{\partial u}{\partial \vec{\nu}} + r_L u = h^L$$

To treat the boundary condition more accurately, we introduce some ghost points outside the domain, we extend the grid by including the indices $-1, N+2$ in the x -direction and $-1, M+2$ in the y -direction. In this way we are able to discretize the problem using central first order FD formulas. On the left side the boundary points are (x_0, y_j) , $j = 1, \dots, M$ and the outward normal is $\vec{\nu} = (-1, 0)$:

$$\begin{aligned} -\frac{u_{1,j} - u_{-1,j}}{2h_x} + r_L u_{0,j} &= h^L(x_0, y_j) \\ u_{-1,j} &= u_{1,j} + 2h_x(h_{0,j}^L - r_L u_{0,j}) \end{aligned}$$

Now substituting $u_{-1,j}$ in (1.2) for $i = 0$ and $j = 1, \dots, M$:

$$\begin{aligned} -\left(\frac{2u_{1,j} - (2 + 2h_x r_L) u_{0,j}}{h_x^2} + \frac{u_{0,j+1} - 2u_{0,j} + u_{0,j-1}}{h_y^2} \right) + \mathbf{b} \cdot \left(\frac{2h_x r_L u_{0,1}}{2h_x}, \frac{u_{0,j+1} - u_{0,j-1}}{2h_y} \right) + \\ + q u_{0,j} = f_{0,j} + \frac{2}{h_x} h_{0,j}^L + \mathbf{b}_1 h_{0,j}^L. \end{aligned}$$

We can notice that in the computation of A the matrices D_y, D_{yy} don't

change because of the outward normal on the left side, instead D_x , D_{xx} and F need to be adapted properly.

Extending this procedure we obtain the FD equations for the right, bottom and top sides.

If we apply the Robin condition on two or more sides, we have to pay attention when discretizing the condition in the corner points, because the normal derivative is not well defined. That's why we apply two directional derivatives in the horizontal and vertical directions in order to get an approximation.

Consider as an example the case in which Robin B.C. is applied both on top and right side, we have to consider the contribution of the right-top corner (x_{N+1}, y_{M+1}) to the discretized system

$$\begin{aligned}
 & - \left(\frac{u_{N,M+1} - 2u_{N+1,M+1} + u_{N+2,M+1}}{h_x^2} + \frac{u_{N+1,M} + 2u_{N+1,M+1} + u_{N+1,M+2}}{h_y^2} \right) + \\
 & + \mathbf{b} \cdot \left(\frac{u_{N+2,M+1} - u_{N,M+1}}{2h_x} + \frac{u_{N+1,M+2} - u_{N+1,M}}{2h_y} \right) + qu_{N+1,M+1} = f_{N+1,M+1}
 \end{aligned}$$

The ghost nodes, obtained applying Robin B.C. to the chosen sides are:

$$\begin{aligned}
 u_{N+2,M+1} &= u_{N,M+1} + 2h_x h_{N+1,M+1}^R - 2r_R h_x u_{N+1,M+1}, \\
 u_{N+1,M+2} &= u_{N+1,M} + 2h_y h_{N+1,M+1}^T - 2r_T h_y u_{N+1,M+1};
 \end{aligned}$$

Substituting these values inside the discretized problem

$$\begin{aligned}
 & \left(\frac{2u_{N,M+1} - 2u_{N+1,M+1}(1 + h_x r_R)}{h_x^2} + \frac{2u_{N+1,M} - 2u_{N+1,M+1}(1 + h_y r_T)}{h_y^2} \right) + \\
 & \mathbf{b} \cdot \left(-r_R u_{N+1,M+1} - r_T u_{N+1,M+1} \right) = f_{N+1,M+1} + \frac{2}{h_x} h_{N+1,M+1}^R + \\
 & + \frac{2}{h_y} h_{N+1,M+1}^T - \mathbf{b}_1 h_{N+1,M+1}^R - \mathbf{b}_2 h_{N+1,M+1}^T.
 \end{aligned}$$

We can proceed exactly as before and modify the matrix A and the vector F relatively to this specific corner. The procedure for the other corners is identical to this one, there is only to adapt to the specific side chosen.

1.2 Accuracy and stability:

Before facing the stability of the problem, let's concentrate on the accuracy.

1.2.1 Accuracy

By Taylor expansion, we can easily compute the local truncation error, obtaining:

$$\tau_{i,j} = \mathcal{L}_h(u_{i,j}) - (-\Delta u + \mathbf{b} \cdot \nabla u + qu) \quad (1.6)$$

with $\mathcal{L}_h$ the finite difference operator associated to our problem, that is

$$\begin{aligned} \mathcal{L}_h = & - \left(\frac{u_{i+1,j} - 2u_{i,j} + u_{i-1,j}}{h_x^2} + \frac{u_{i,j+1} - 2u_{i,j} + u_{i,j-1}}{h_y^2} \right) + \\ & \mathbf{b}_1 \frac{u_{i+1,j} - u_{i-1,j}}{2h_x} + \mathbf{b}_2 \frac{u_{i,j+1} - u_{i,j-1}}{2h_y} + qu_{i,j}. \end{aligned} \quad (1.7)$$

By Taylor expansion of $u(x_i + h, y_j)$ and $u(x_i - h, y_j)$ both centered in (x_i, y_j) , we get:

$$\begin{aligned} u(x_i + h_x, y_j) = & u(x_i, y_j) + h_x \frac{\partial u(x_i, y_j)}{\partial x} + \frac{h_x^2}{2} \frac{\partial^2 u(x_i, y_j)}{\partial x^2} + \\ & \frac{h_x^3}{6} \frac{\partial^3 u(x_i, y_j)}{\partial x^3} + \frac{h_x^4}{24} \frac{\partial^4 u(x_i, y_j)}{\partial x^4} + \mathcal{O}(h_x^4) \end{aligned} \quad (1.8)$$

$$\begin{aligned} u(x_i - h_x, y_j) = & u(x_i, y_j) - h_x \frac{\partial u(x_i, y_j)}{\partial x} + \frac{h_x^2}{2} \frac{\partial^2 u(x_i, y_j)}{\partial x^2} - \\ & + \frac{h_x^3}{6} \frac{\partial^3 u(x_i, y_j)}{\partial x^3} + \frac{h_x^4}{24} \frac{\partial^4 u(x_i, y_j)}{\partial x^4} + \mathcal{O}(h_x^4) \end{aligned} \quad (1.9)$$

Using that $x_i - h_x = x_{i-1}$ and $x_i + h_x = x_{i+1}$, we get from (1.8)+(1.9) the Taylor expansion of the second order terms:

$$\begin{aligned} u(x_{i+1}, y_j) + u(x_{i-1}, y_j) = & 2u(x_i, y_j) + h_x^2 \frac{\partial^2 u(x_i, y_j)}{\partial x^2} \\ & + \frac{h_x^4}{12} \frac{\partial^4 u(x_i, y_j)}{\partial x^4} + \mathcal{O}(h_x^4) \end{aligned} \quad (1.10)$$

and, with some adjustments,

$$\frac{u(x_{i+1}, y_j) - 2u(x_i, y_j) + u(x_{i-1}, y_j))}{h_x^2} - \frac{\partial^2 u(x_i, y_j)}{\partial x^2} = + \frac{h_x^2}{12} \frac{\partial^4 u(x_i, y_j)}{\partial x^4} + \mathcal{O}(h_x^4) \quad (1.11)$$

Then, by denoting $u_{i,j} := u(x_i, y_j)$ we have

$$\frac{u_{i+1,j} - 2u_{i,j} + u_{i-1,j}}{h_x^2} - \frac{\partial^2 u_{i,j}}{\partial x^2} = \mathcal{O}(h_x^2) \quad (1.12)$$

By the same reasoning, by Taylor expansion on $u(x_i, y_j + h_y)$ and on $u(x_i, y_j - h_y)$ we obtain

$$\frac{u_{i,j+1} - 2u_{i,j} + u_{i,j-1}}{h_y^2} - \frac{\partial^2 u_{i,j}}{\partial y^2} = \mathcal{O}(h_y^2) \quad (1.13)$$

In a similar way we get the Taylor expansion of the first order terms, by subtracting (1.8) and (1.9):

$$\begin{aligned} u(x_{i+1}, y_j) - u(x_{i-1}, y_j) &= 2h_x \frac{\partial u(x_i, y_j)}{\partial x} \\ &+ 2 \frac{h_x^3}{6} \frac{\partial^3 u(x_i, y_j)}{\partial x^3} + \mathcal{O}(h_x^3) \end{aligned} \quad (1.14)$$

which implies

$$\frac{u_{i+1,j} - u_{i-1,j}}{2h_x} - \frac{\partial u_{i,j}}{\partial x} = \mathcal{O}(h_x^2) \quad (1.15)$$

Analogously for the y component:

$$\frac{u_{i,j+1} - u_{i,j-1}}{2h_y} - \frac{\partial u_{i,j}}{\partial y} = \mathcal{O}(h_y^2) \quad (1.16)$$

Now, substituting (1.15), (1.16), (1.12), (1.13) inside (1.6) the following accuracy result follows in a practically direct way:

$$\tau_{i,j} = \mathcal{O}(h_x^2 + h_y^2)$$

In our Matlab App we'll have $h_x \approx h_y$ as the domain mesh refines, thus we obtain $\tau_{i,j} = \mathcal{O}(h^2)$: the method is second order accurate.

We consequently expect to plot the error vs the step h_x as a straight line parallel to h_x^2 in a logarithmic scale inside our Matlab App.

1.2.2 Stability

The diffusion-convection-reaction problem can be rewritten in terms of the derivative of $u(x, y)$ with respect to time as:

$$\frac{\partial u}{\partial t} - \Delta u + \mathbf{b} \cdot \nabla u + qu = f$$

The study of the stability can be approached considering the homogeneous version of the time-dependent problem

$$\frac{\partial u}{\partial t} = \Delta u - \mathbf{b} \cdot \nabla u - qu$$

The discretization of the problem, through the five-points stencil is:

$$\begin{aligned} \frac{u_{i,j}^{n+1} - u_{i,j}^n}{\Delta t} = & \left(\frac{u_{i-1,j}^n - 2u_{i,j}^n + u_{i+1,j}^n}{\Delta x^2} + \frac{u_{i,j-1}^n - 2u_{i,j}^n + u_{i,j+1}^n}{\Delta y^2} \right) + \\ & - \mathbf{b} \cdot \left(\frac{u_{i+1,j}^n - u_{i-1,j}^n}{2\Delta x}, \frac{u_{i,j+1}^n - u_{i,j-1}^n}{2\Delta y} \right) - qu_{i,j}^n \end{aligned}$$

The stability can be studied through the Von-Neumann analysis, also known as Fourier stability analysis which is based on the substitution of the Fourier mode into the discretized problem

$$u_{i,j}^n = \hat{u}^n e^{ik(x_i + y_j)}$$

where $\hat{u}^n$ is the amplitude of the mode at time level n , k is the wavenumber, i is the complex number and x_i, y_j are the grid points at the position i -th and j -th.

Substituting the Fourier mode into the discretized equation and simplifying:

$$\begin{aligned} \hat{u}^{n+1} = \hat{u}^n \left[1 + \frac{2\Delta t}{\Delta x^2}(\cos(k\Delta x) - 1) + \frac{2\Delta t}{\Delta y^2}(\cos(k\Delta y) - 1) - i\mathbf{b} \cdot \right. \\ \left. \left(\frac{\Delta t}{\Delta x} \sin(k\Delta x), \frac{\Delta t}{\Delta y} \sin(k\Delta y) \right) - \Delta tq \right] \end{aligned}$$

To ensure the stability we require that:

$$\begin{aligned} |G| = \left| \frac{\hat{u}^{n+1}}{\hat{u}^n} \right| = \left| 1 + \frac{2\Delta t}{\Delta x^2}(\cos(k\Delta x) - 1) + \frac{2\Delta t}{\Delta y^2}(\cos(k\Delta y) - 1) - i\mathbf{b} \cdot \right. \\ \left. \left(\frac{\Delta t}{\Delta x} \sin(k\Delta x), \frac{\Delta t}{\Delta y} \sin(k\Delta y) \right) - \Delta tq \right| \end{aligned}$$

Let:

$$\begin{aligned} A &= \frac{2\Delta t}{\Delta x^2}(\cos(k\Delta x) - 1) + \frac{2\Delta t}{\Delta y^2}(\cos(k\Delta y) - 1) - \Delta tq \\ B &= -\mathbf{b} \cdot \left(\frac{\Delta t}{\Delta x} \sin(k\Delta x), \frac{\Delta t}{\Delta y} \sin(k\Delta y) \right) \end{aligned}$$

Assuming as in our MatlabApp that $\Delta x \approx \Delta y$

$$\begin{aligned} A &= \frac{4\Delta t}{\Delta x^2}(\cos(k\Delta x) - 1) - \Delta tq \\ B &= -\mathbf{b} \cdot \left(\frac{\Delta t}{\Delta x} \sin(k\Delta x), \frac{\Delta t}{\Delta x} \sin(k\Delta x) \right) \end{aligned}$$

CHAPTER 1. CENTRAL FD FOR DIFFUSION-CONVECTION-REACTION9

where A is the Diffusion-Reaction term, while B is the convection term. For the stability we require that:

$$|(1 + A) + iB| \leq 1 \implies \sqrt{(1 + A)^2 + B^2} \leq 1$$

Let's analyze the terms A and B :

1. Diffusion-reaction term A

- $\frac{4\Delta t}{\Delta x^2}(\cos(k\Delta x) - 1)$ ranges from $\frac{-8\Delta t}{\Delta x^2}$ when $\cos(k\Delta x)$ is equal to -1, to 0 when $\cos(k\Delta x)$ is equal to 1

thus:

$$\frac{-8\Delta t}{\Delta x^2} - \Delta tq \leq A \leq \Delta tq \implies 1 - \frac{8\Delta t}{\Delta x^2} - \Delta tq \leq 1 + A \leq 1 - \Delta tq$$

2. Convection term B

- The term B ranges from $-\mathbf{b} \cdot \left(\frac{\Delta t}{\Delta x}, \frac{\Delta t}{\Delta x}\right)$ to $\mathbf{b} \cdot \left(\frac{\Delta t}{\Delta x}, \frac{\Delta t}{\Delta x}\right)$

To ensure the stability condition we require that:

$$\left(1 - \frac{8\Delta t}{\Delta x^2} - \Delta tq\right)^2 + \left(\mathbf{b} \cdot \left(\frac{\Delta t}{\Delta x}, \frac{\Delta t}{\Delta x}\right)\right)^2 \leq 1$$

The conditions for Δt ensuring stability are:

$$\Delta t \leq \frac{2\Delta x^2}{8 + q\Delta x^2} \quad \Delta t \leq \frac{\Delta x}{\min(|b_1|, |b_2|)}$$

that can be rewritten as

$$\Delta t \leq \min\left(\frac{2\Delta x^2}{8 + q\Delta x^2}, \frac{\Delta x}{\min(|b_1|, |b_2|)}\right)$$

This ensures that G remains bounded, indicating the stability of the problem.

Chapter 2

A brief guide on how the app works

We decided to divide the two modalities using two different panels. On the tool bar on the top there are the "set app mode", which allows us to change modality, the "set problem parameters", in which we can choose the vector $\mathbf{b}$ and q and the "set domain", where we can choose the extremes of the rectangular domain. The only difference on the tool bar, between the two modalities, is the "set boundary condition". In the first modality we are only able to choose where to apply the Robin and Dirichlet conditions, instead in the second one we have to specify the value of g_D , that indicates the Dirichlet condition and g_R , g_L , g_T , g_B that indicate the Robin conditions applied on the sides.

In order to explain the project we need to do some specification on the notation we used:

- B_x and B_y are the components of the vector $\mathbf{b}$ related to the first derivative of u with respect to (x, y) ;
- q is the coefficient of u ;
- a , b , c , d are the extremes of the rectangular domain.

2.1 First modality:

In the first modality of Matlab App, the user is able to set the exact solution u , the problem parameters, the problem domain and the boundary condition. Once all the parameters are set it's possible to choose to:

- Visualize the convergence plot;
- Visualize the matrix conditioning number;
- Visualize the numerical solution.

In the convergence plot and the matrix conditioning number we decided to add a table with the values of N (number of grid points in the x -direction), M (number of grid points in the y -direction), h_x , h_y , the infinity norm of the error and the conditioning number of the matrix.

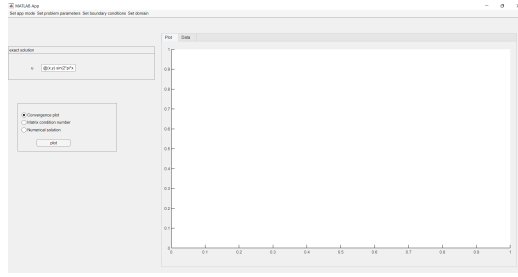


Figure 2.1: Layout of the first version of the App

Example 1. Consider the problem on the domain $\Omega = [0, 1] \times [0, 1]$, with homogeneous Dirichlet boundary conditions.

$$\begin{cases} -\Delta u = f & (x, y) \in \Omega \\ u = 0 & (x, y) \in \partial\Omega \end{cases}$$

where our exact solution is $u(x, y) = \sin(2\pi x) \sin(2\pi y)$.

Notice that $\mathbf{B} = (0, 0)$ and $q = 0$.

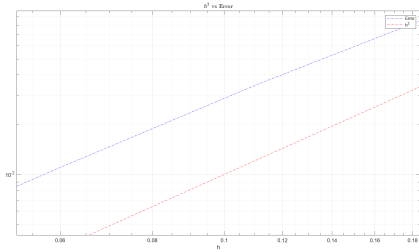


Figure 2.2: Convergence plot

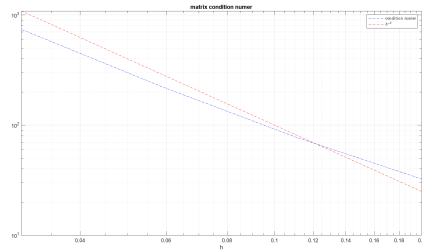


Figure 2.3: Conditioning

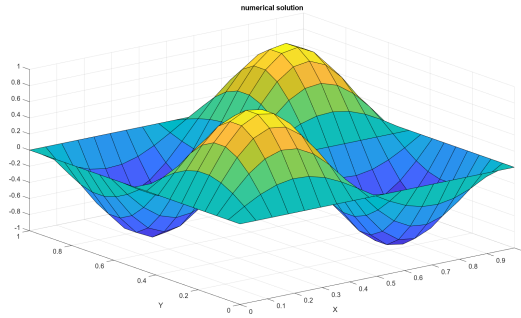


Figure 2.4: Numerical solution

Example 2. Consider the problem on the domain $\Omega = [0, 1] \times [0, 1]$, with homogeneous Dirichlet boundary conditions on the left side and Robin boundary conditions on the remaining boundary.

$$\begin{cases} -\Delta u + \pi u = f & (x, y) \in \Omega \\ u = 0 & (x, y) \in \Gamma_D \\ \frac{\partial u}{\partial \nu} + ru = gK & (x, y) \in \Gamma_K, \quad K \in \{R, B, T\}, \quad r = 1.5. \end{cases}$$

where our exact solution is $u(x, y) = \sin(2x)(x^2 + y^2)$.

Notice that $\mathbf{B} = (0, 0)$ and $q = \pi$.

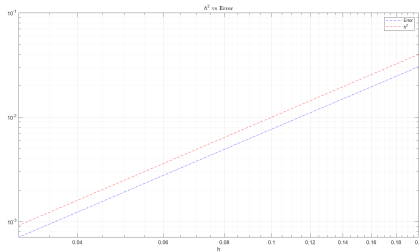


Figure 2.5: Convergence plot

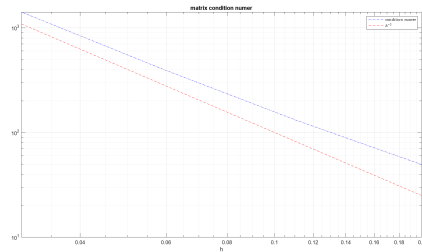


Figure 2.6: Conditioning

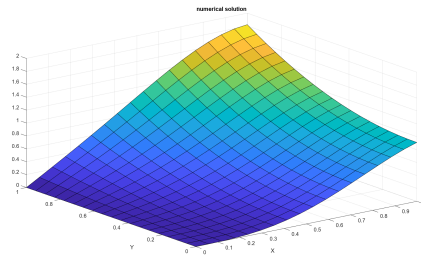


Figure 2.7: Numerical solution

Example 3. Consider the Dirichlet problem of the first example with $\mathbf{B} = (2, 1.5)$ and $q = 1$. All the other parameters remain the same as in the previous example.

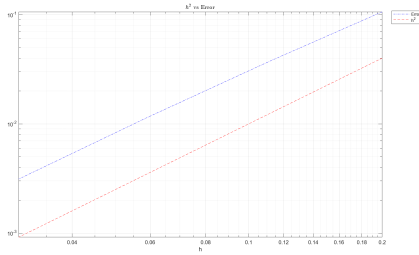


Figure 2.8: Convergence plot

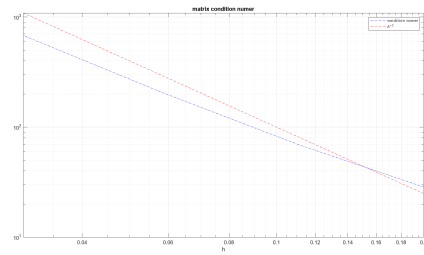


Figure 2.9: Conditioning

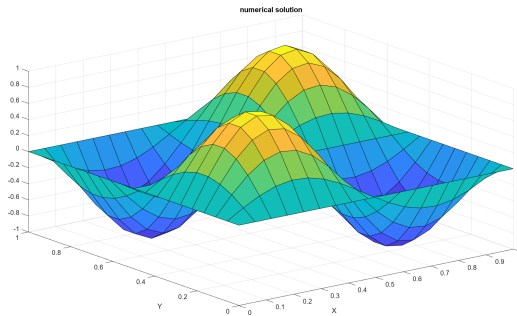


Figure 2.10: Numerical solution

We notice that the error is of an higher order with respect to the first example and the matrix conditioning number has a similar behaviour to the matrix associated to the Poisson problem with homogeneous Dirichlet condition.

2.2 Second modality:

In the second modality of our Matlab App, you can set problem parameters $\mathbf{b}$ and q , the problem domain exactly as in the first version.

Regarding the boundary conditions, in this case you can set them more freely: you can choose between Dirichlet and Robin (along with the sides on which to apply them) as previously, but you can in addition specify the functions g_D, g_L, g_R, g_T, g_B , defined taking advantage on the notation introduced in chapter 1:

- g_D such that $u = g_D$ on Γ_D
- g_L, g_R, g_T, g_B such that $\frac{\partial u}{\partial \nu} + ru = gK$ on $\Gamma_K, K \in \{L, R, T, B\}$

In the central, left part of the panel, it's possible to choose the grid dimensions in both x and y direction (N and M) respectively along with f , which refers to the problem (1.1). This version of the app simply aims to plot the numerical

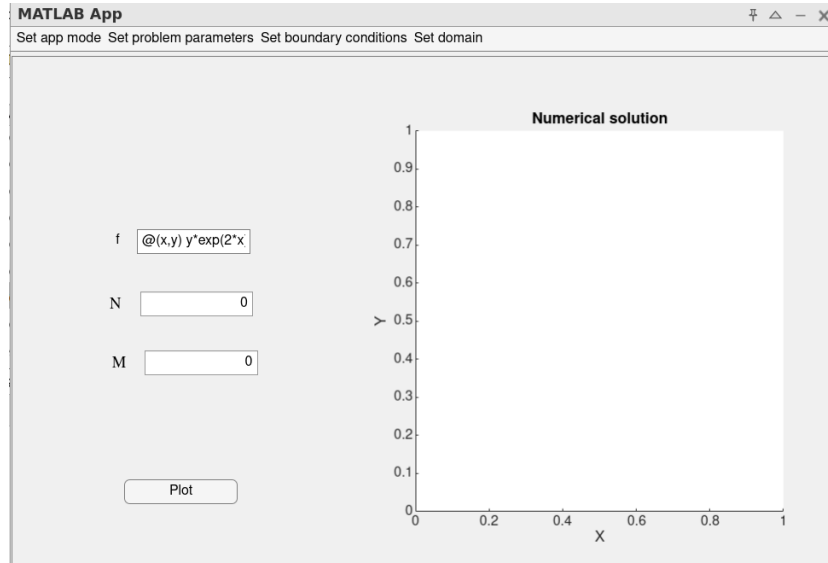


Figure 2.11: UI of the second version of the App

solution of the problem (1.1) with the specified problem parameters, domain settings and grid dimensions, computed through FD method using the mixed boundary conditions fixed by the user themselves. Once the environment is set up, users simply need to click the 'Plot' button to activate the app.

We'll show here some examples.

Example 1: Elliptic problem on the domain $\Omega = [-\pi/2, \pi/2] \times [-\pi, 0]$, with homogeneous Dirichlet Boundary conditions.

$$\begin{cases} -0.4\Delta u = f & (x, y) \in \Omega, \quad k = 0.4 \\ u = 0 & (x, y) \in \partial\Omega \end{cases}$$

where

$$f(x, y) = -\left(\frac{2 \exp(x) \cos(x) \cos(y) \sin(y)}{5}\right) - \left(\frac{4 \exp(x) \sin(x) \sin(y)}{5}\right)$$

The true solution is:

$$u(x, y) = -\exp(x) \cos(x) \sin(y)$$

We can assess this problem rewriting it as

$$-\Delta u = f/0.4$$

thus inserting $f/0.4$ in our text application Edit Field. We use the app to compute the numerical solution:

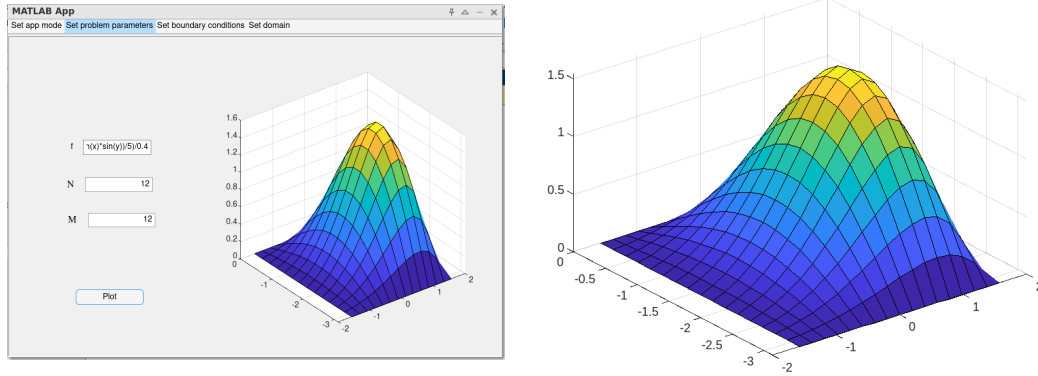


Figure 2.12: Screenshots from the app showing the UI after the plot on the left, the numerical solution on the right choosing $N = M = 15$

Example 2 Consider the problem

$$\begin{cases} -\Delta u + \frac{\partial u}{\partial x} + \pi u = f & (x, y) \in \Omega \\ u = 3 & (x, y) \in \Gamma_D = \Gamma_R \\ \frac{\partial u}{\partial \nu} + ru = gK & (x, y) \in \Gamma_K, \quad K \in \{L, T, B\} \quad r = 1.5 \end{cases}$$

with

- $f = @(x,y)y * \exp(2 * x) - 7 * x * \exp(2 * x) - 4 * \exp(2 * x) - 3 * \exp(2 * x) * (x^2 + y^2) + pi * \exp(2 * x) * (x^2 + y^2)$
- $gL = @(x,y) - 2 * x * \exp(2 * x) - (\exp(2 * x) * (x^2 + y^2))/2$
- $gB = @(x,y)(3 * \exp(2 * x) * (x^2 + y^2))/2 - 2 * y * \exp(2 * x)$
- $gT = @(x,y)2 * y * \exp(2 * x) + (3 * \exp(2 * x) * (x^2 + y^2))/2$

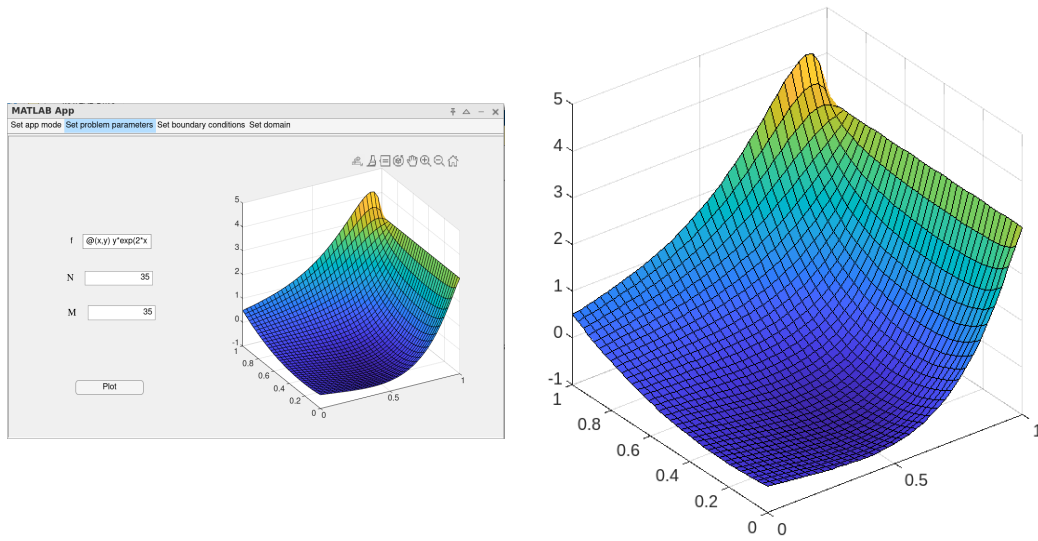


Figure 2.13: Screenshots from the app showing the UI after the plot on the left, the numerical solution on the right choosing $N = M = 35$

Chapter 3

Conclusions

We can observe that the numerical results are consistent with the theoretical expectations discussed during the course and the analysis done in the chapter regarding the accuracy and stability of the Diffusion-Convection-Problem. Furthermore either we choose the exact solution $u(x, y)$ of the problem, as requested in the version one of the App, or we give the known data f , version two of the App, the results are consistent in both ways.